Chapter 9. Functional Programming Techniques: Functional Interfaces, Streams, and Parallel Collections

9.0 Introduction

Java is an object-oriented (OO) language. You know what that is. Functional programming (FP) has been attracting attention lately. There may not be quite as many definitions of FP as there are FP languages, but it's close. Wikipedia's definition of functional programming is as follows (viewed December 2013):

> *a programming paradigm, a style of building the structure and elements of computer programs, that treats computation as the evaluation of mathematical functions and avoids state and mutable data. Functional programming emphasizes functions that produce results that depend only on their inputs and not on the program state—i.e. pure mathematical functions. It is a declarative programming paradigm, which means programming is done with expressions. In functional code, the output value of a function depends only on the arguments that are input to the function, so calling a function f twice with the same value for an argument x will produce the same result f(x) both times. Eliminating side effects, i.e. changes in state that don't depend on the function inputs, can make it much easier to understand and predict the behavior of a program, which is one of the key motivations for the development of functional programming.*

How can we benefit from the FP paradigm? One way would be to switch to using an FP language; some of the leading languages are Haskell,[1] Idris, OCaml, Erlang, Julia, and the LISP family. But most of those would require walking away from the Java ecosystem. You could consider using Scala or Clojure, JVM-based languages that provide functional programming support in the context of an OO language.

But this is the *Java Cookbook*, so as you can imagine, we're going to try to get as many benefits of functional programming as we can while remaining in the Java language. Some features of FP include the following:

- Pure functions that have no side effects and whose results depend only on their inputs and not on mutable state elsewhere in the program
- First-class functions (e.g., functions as data)
- Immutable data
- Extensive use of recursion and lazy evaluation

Pure functions are completely self-contained; their operation depends only on the input parameters and internal logic, not on any variable state in other parts of the program—indeed, there are no global variables, only global *constants*. Although this can be hard to accept for those schooled in imperative languages like Java, it does make it much easier to test and ensure program correctness! It means that, no matter what else is going on in the program (even with multiple threads), a method call like `computeValue(27)` will always, unconditionally, return the same value every time (with exceptions, of course, for things like the current time, random seeds, etc., which are global state).

We'll use the terms *function* and *method* interchangeably in this chapter, although this is not strictly correct. FP people use the term *function* in the mathematical function sense, whereas in Java *methods* just means some code you can call (a Java method call is also referred to as a *message* being *sent* to an object, in the OO view of things).

Functions as data means that you can create *an object that is a function*, pass it into another function, write a function that returns another function, and so on—with no special syntax, because, well, functions *are* data.

One of Java's approaches to FP is the definition of functional interfaces. A *functional interface* in Java is one that has only one abstract method, such as the widely used `Runnable`, whose only method is `run()`, or the common Swing action handler `ActionListener`, whose only method is `actionPerformed(ActionEvent)`. As we saw in Recipe 8.7, Java 8+ allows interfaces to have `default` methods, which are available for use in any class that `implements` the interface. Such methods cannot depend on instance state in a particular class because they would have no way of referring to it at compile time.

So a functional interface is more precisely defined as one that has a single nondefault method. You can do functional-style programming in Java if you use functional interfaces and if you restrict code in your methods to not depending on any nonfinal instance or class fields; using `default` methods is one way of achieving this. The first few recipes in this chapter discuss functional interfaces.

Another Java approach to functional-ness is the *lambda expression*. A lambda is an expression of a functional interface, and it can be used as data (i.e., assigned, returned, etc.). Just to give a couple of short examples for now:

```
ActionListener x = e -> System.out.println("You activated " + e.getSource());
```

```
public class RunnableLambda {

 private static ExecutorService threadPool =
  Executors.newSingleThreadExecutor();

 public static void main(String[] args) {
  threadPool.submit(() -> System.out.println("Hello from a thread"));
```

Immutable data is easy in theory: just have a class with only read accessors (`get` methods). The downside is it takes some getting used to. But it is worthwhile and it works. The standard `String` class, for example, is immutable: methods like `substring()` or `toUpperCase()` don't change the original string, but make up new string objects with the requested change. Yet strings are universally used, and useful. Enums are also implicitly immutable. Java 16 added a new kind of class-like object called a `record`; `record`s are implicitly immutable. The compiler generates `get`-like methods for the fields (along with a constructor and the common `Object` methods), but no `set` methods.

Java 8 also introduced `Stream` classes. A `Stream` is like a pipeline that you can feed into, fan out, collect down—like a cross between the Unix notion of pipelines and Google's distributed programming concept of MapReduce, as exemplified in Hadoop, but running in a single program in a single VM. `Stream`s can be sequential or parallel; the latter are designed to take advantage of the massive parallelism that is happening in hardware design (particularly servers, where 16-or-more-core processors are popular). We discuss `Stream`s in several recipes in this chapter.

If you're familiar with Unix pipes and filters, this equivalence will make sense to you; if not, you can skip it for now. The Unix command is this:

```
cat lines.txt | sort | uniq | wc -l
```

The Java `Stream`s equivalent is shown in Example 9-1. For small inputs, the Unix pipeline is faster; but for larger volumes, the Java one should be faster, especially when

parallelized.

Example 9-1. main/src/main/java/functional/UnixPipesFiltersReplacement.java

```
    long numberLines = Files.lines(Path.of(("lines.txt")))
      .sorted()
      .distinct()
      .count();
    System.out.printf("lines.txt contains " + numberLines + " unique lines.");
```

Tied in with `Stream`s is the notion of a `Spliterator`, a derivative (logically, not by inheritance) of the familiar `Iterator` but designed for use in parallel processing. Most users will not be expected to develop their own `Spliterator` and will likely not even call its methods directly very often, so we do not discuss it in detail.

See Also

For general information on functional programming, see the book *Functional Thinking* by Neal Ford (O'Reilly).

There is an entire book dedicated to lambda expressions and related tools, Richard Warburton's *Java 8 Lambdas* (O'Reilly).

9.1 Using Lambdas/Closures Instead of Inner Classes

Problem

You want to avoid all the typing that even the anonymous style of inner class requires.

Solution

Use Java's lambda expressions.

Discussion

The symbol lambda (λ) is the 11th letter of the Greek alphabet and thus as old as Western society. The Lambda calculus is older than our notions of computing. In this

context, lambda expressions are small units of calculation that can be referred to. They are functions as data. In that sense, they are a lot like anonymous inner classes, though it's probably better to think of them as *anonymous methods*. They are essentially used to replace inner classes for a *functional interface*—that is, an interface with one abstract method (function) in it. A desktop example is the AWT `ActionListener` interface, widely used in GUI code, whose only method is this one:

```
public void actionPerformed(ActionEvent e);
```

Using lambdas is now the preferred method of writing for GUI action listeners. Here's a single example:

```
quitButton.addActionListener(e -> shutDownApplication(0));
```

Closures in computing are functions that capture (have access to) variables from their surrounding environment. This allows them to access and modify variables from outside their scope. Closures are not explicitly supported in Java, but can be simulated using lambdas with effectively final variables.

Because fewer developers write Swing GUI applications these days, let's start with an example that doesn't require GUI programming. Suppose we have a collection of camera model descriptor objects that has already been loaded from a database into memory, and we want to write a general-purpose API for searching them, for use by other parts of our application.

The first thought might be along the following lines:

```
public interface CameraInfo {
   public List<Camera> findByMake();
   public List<Camera> findByModel();
   ...
}
```

Perhaps you can already see the problem. You will also need to write `findByPrice()`, `findByMakeAndModel()`, `findByYearIntroduced()`, and so on as your application grows in complexity.

You could consider implementing a query by example method, where you pass in a `Camera` object and all its nonnull fields are used in the comparison. But then how would you implement finding cameras with interchangeable lenses *under $500*?[2]

So a better approach is probably to use a callback function to do the comparison. Then you can provide an anonymous inner class to do any kind of searching you need. You'd want to be able to write callback methods like this:

```
public boolean choose(Camera c) {
   return c.isIlc() && c.getPrice() < 500;
}
```

Accordingly, we'll build that into an interface:[3]

```
/** An Acceptor accepts some elements from a Collection */
public interface CameraAcceptor {
   boolean choose(Camera c);
}
```

Now the search application provides a method:

```
public List<Camera> search(CameraAcceptor acc);
```

which we can call with code like this:

```
results = searchApp.search(new CameraAcceptor() {
   public boolean choose(Camera c) {
      return c.isIlc() && c.getPrice() < 500;
   }
}
```

Or, if you are not comfortable with anonymous inner classes, you might have to type this:

```
class MyIlcPriceAcceptor implements CameraAcceptor {
   public boolean choose(Camera c) {
      return c.isIlc() && c.getPrice() < 500;
   }
}
```

```
CameraAcceptor myIlcPriceAcceptor = new MyIlcPriceAcceptor();
results = searchApp.search(myIlcPriceAcceptor);
```

That's really a great deal of typing just to get one method packaged up for sending into the search engine. Java's support for lambda expressions or closures was argued about for many years (literally) before the experts agreed on how to do it. And the result is staggeringly simple. One way to think of Java lambda expressions is that each one is just a method that implements a functional interface. With lambda expressions, you can re-write the preceding code as just:

```
results = searchApp.search(c -> c.isIlc() && c.getPrice() < 500);
```

The arrow notation `->` indicates the code to execute. If it's a simple expression, as we have here, you can just write it as shown. If there is conditional logic or other statements, you have to use a block, as is usual in Java.

Here is the search example rewritten (more verbosely than needed) to show how it would look as a block:

```
results = searchApp.search(c -> {
   if (c.isIlc() && c.getPrice() < 500)
      return true;
   else
      return false;
});
```

The first `c` inside the parenthesis corresponds to `Camera c` in the explicitly implemented `choose()` method: you can omit the type because the compiler knows it! If there is more than one argument to the method, you must parenthesize them. Suppose we had a compare method that takes two cameras and returns a quantitative value (oh, and good luck trying to get two photographers to agree on *that* algorithm!):

```
double goodness = searchApp.compare((c1, c2) -> {
   // write some amazing code here
});
```

This notion of *lambdas* seems pretty potent, and it is! You will see much more of this in Java as the use of lambdas moves into the mainstream of computing.

Up to this point, we still have to write an interface for each type of method that we want to be able to lambda-ize. The next recipe shows some predefined interfaces that you can use to further simplify (or at least shorten) your code.

And, of course, there are many existing interfaces that are functional, such as the `ActionListener` interface from GUI applications. Interestingly, the IntelliJ IDE (see Recipe 1.4) automatically recognizes inner class definitions that are replaceable by lambdas and, when using *code folding* (the IDE feature that represents an entire method definition with a single line), displays the inner class as the corresponding lambda, but leaves it as an inner class in the code. Modern versions will also offer to replace the entire inner class definition with a lambda when you mouse over the class name in `new ActionListener...`. Figures 9-1 and 9-2 show a before-and-after picture of this code folding. Figure 9-3 shows the IDE offering to replace the inner class with a lambda.

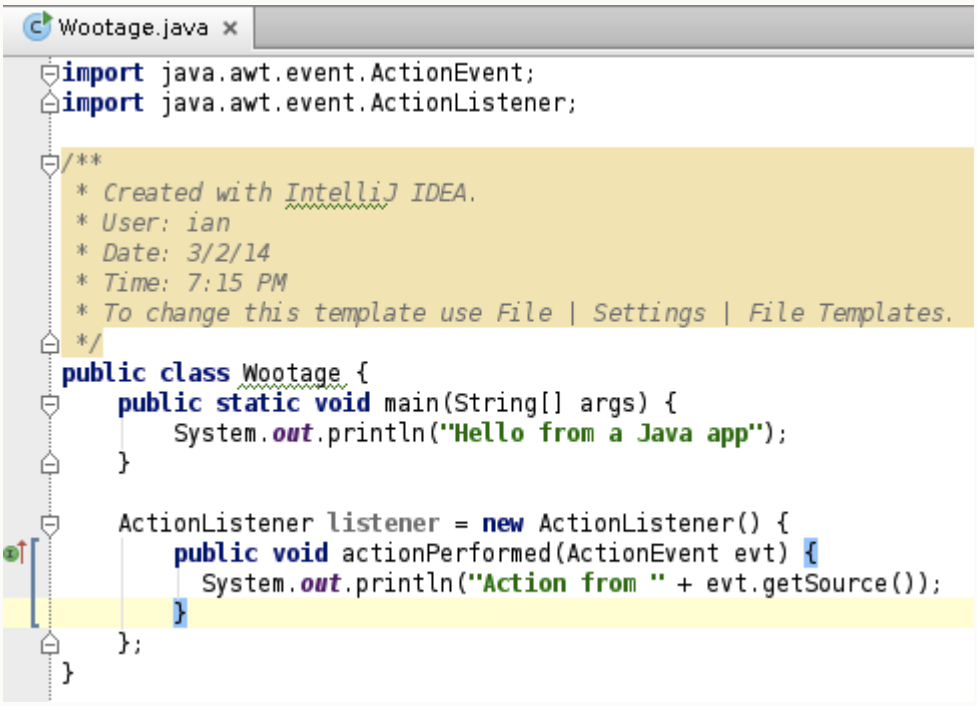


Figure 9-1. IntelliJ code unfolded

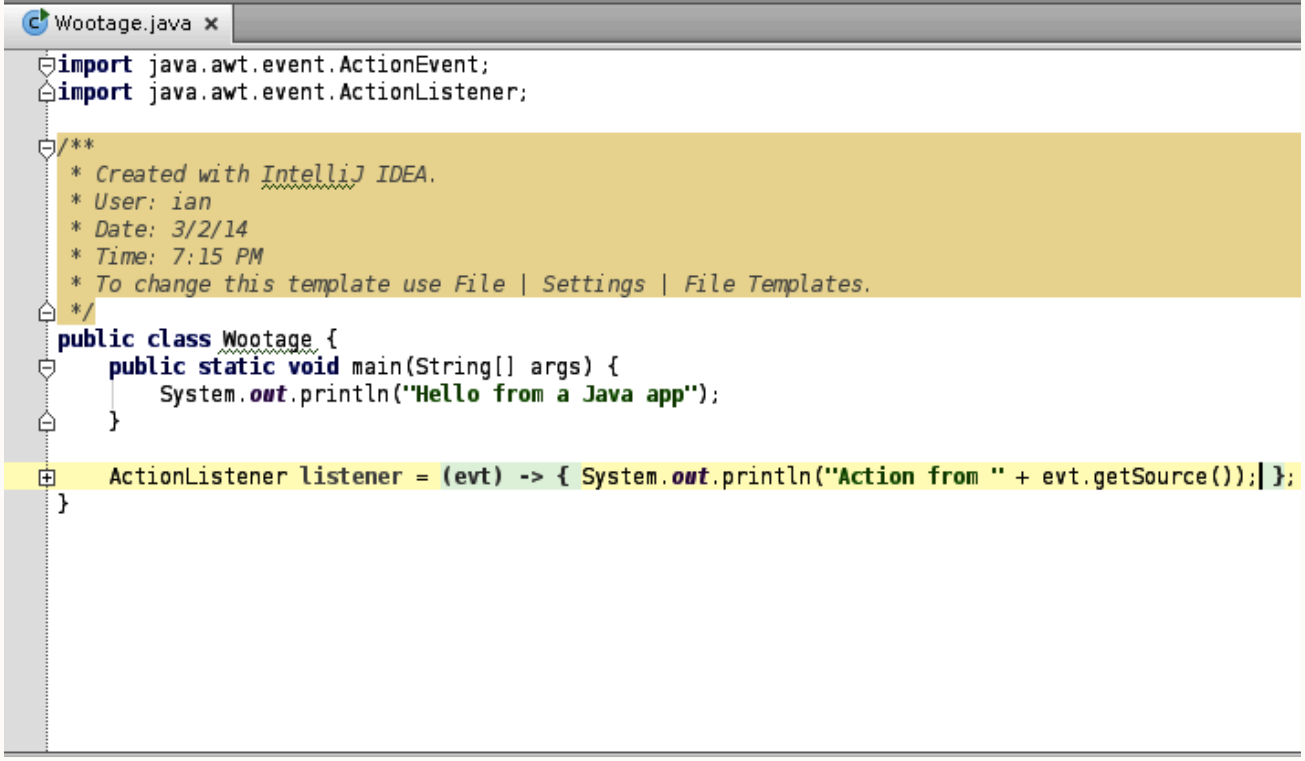


Figure 9-2. IntelliJ code folded

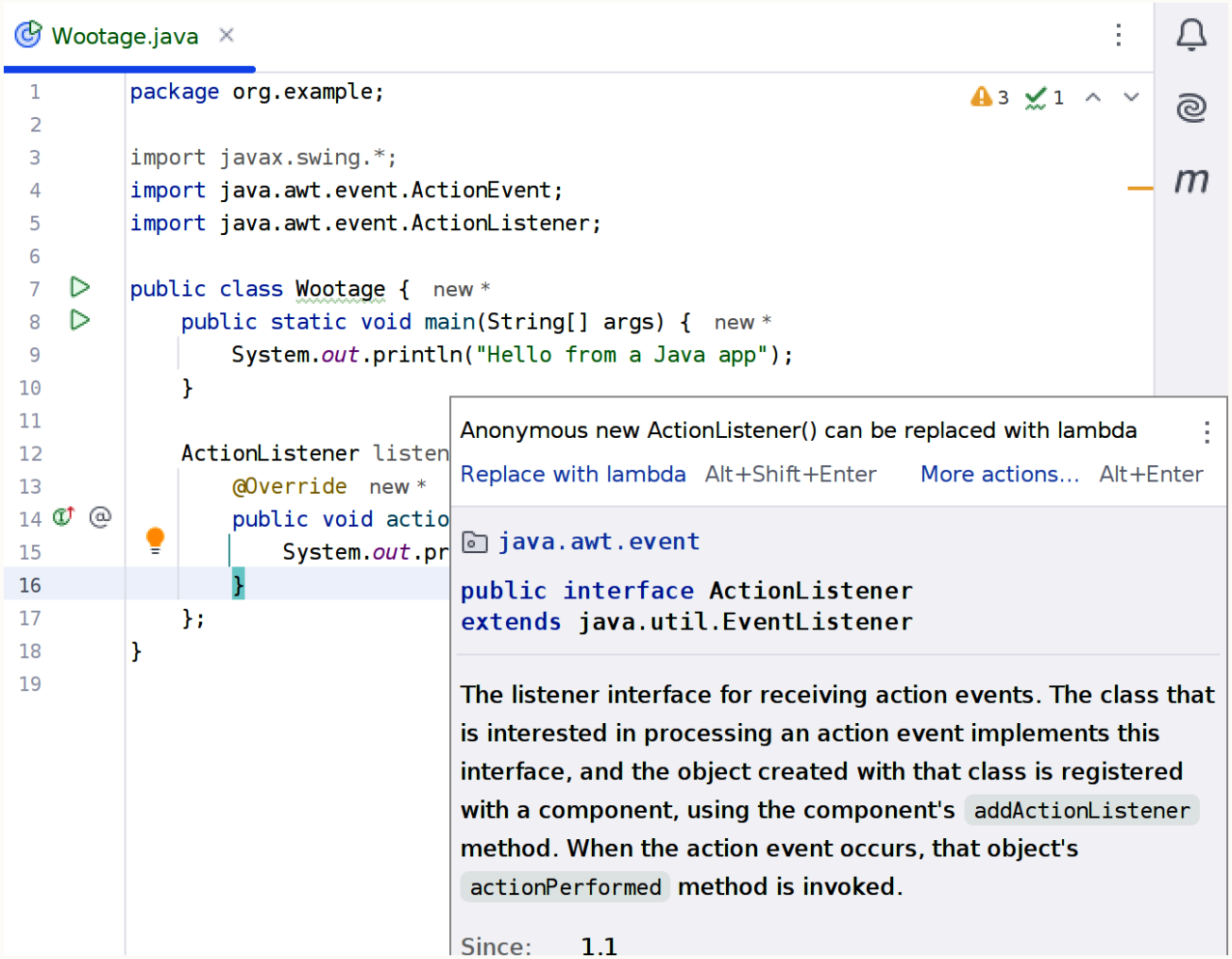


Figure 9-3. IntelliJ offering lambda refactoring

9.2 Using Predefined Lambda Interfaces or

Rolling Your Own

Problem

You want to use existing interfaces, instead of defining your own, for use with lambdas.

Solution

Use the Java 8 lambda functional interfaces from `java.util.function`.

Discussion

In Recipe 9.1, we used the interface method `acceptCamera()` defined in the interface `CameraAcceptor`. Acceptor-type methods are quite common, so the package java.util.function includes the `Predicate<T>` interface, which we can use instead of `CameraAcceptor`. This interface has only one method—`boolean test(T t)`:

```
interface Predicate<T> {
   boolean test(T t);
}
```

This package includes about 50 of the most commonly needed functional interfaces, such as `IntUnaryOperator`, which takes one `int` argument and returns an `int` value; `LongPredicate`, which takes one `long` and returns `boolean`; and so on.

To use the `Predicate` interface, as with any generic type, we provide an actual type for the parameter `Camera`, giving us (in this case) the parameterized type `Predicate<Camera>`, which is the following (although we don't have to write this out):

```
interface Predicate<Camera> {
   boolean test(Camera c);
}
```

So now our search application will be changed to offer us the following search method:

```
public List<Camera> search(Predicate p);
```

Conveniently, this has the same signature as our own `CameraAcceptor` from the point of view of the anonymous methods that lambdas implement, so the rest of our code doesn't have to change! This is still a valid call to the `search()` method:

```
results = searchApp.search(c -> c.isIlc() && c.getPrice() < 500);
```

Here is the implementation of the `search` method:

main/src/main/java/functional/CameraSearchPredicate.java

```
public List<Camera> search(Predicate<Camera> tester) {
   List<Camera> results = new ArrayList<>();
   privateListOfCameras.forEach(c -> {
      if (tester.test(c))
         results.add(c);
   });
   return results;
}
```

Suppose we only need the list to do one operation on each element, and then we'll discard it. Upon reflection, we don't actually need to get the list back; we merely need to get our hooks on each element that matches our `Predicate` in turn.

Roll Your Own Functional Interface

While the JDK provides a good set of functional interfaces, there may be cases where you'd want to create your own. This is a simple example of a functional interface:

```
@FunctionalInterface
interface MyFunctionalInterface {
   int compute(int x);
}
```

The `@FunctionalInterface` annotation tells the compiler to ensure that a given interface is and remains functional. Its use is analogous to `@Override` (both annotations are in `java.lang`). It is always optional.

`MyFunctionalInterface` could be used to process an array of integers, like this:

```
  static int[] integers = {1, 2, 3};

  public static void main(String[] args) {
   int total = 0;
   for (int i : integers)
    total += process(i, x ->  x * x + 1);
   System.out.println("The total is " + total);
  }

  private static int process(int i, MyFunctionalInterface o) {
   return o.compute(i);
  }
```

If `compute` were a nonfunctional interface—having multiple abstract methods—you would not be able to use it in this fashion.

Sometimes, of course, you really do need an interface to have more than one method. In that case, the illusion (or the effect) of functionality can sometimes be preserved by denoting all but one of the methods with the `default` keyword—the nondefault method will still be usable in lambdas. A `default` method has a method body:

```
public interface ThisIsStillFunctional {
   default int compute(int ix) { return ix * ix + 1 };
   int anotherMethod(int y);
}
```

Only `default` methods may contain executable statements, and there may only be one nondefault method per functional interface.

By the way, the `MyFunctionalInterface` given earlier can be totally replaced by `java.util.function.IntUnaryOperator`, changing the method name `apply()` to `applyAsInt()`. There is a version of the `ProcessInts` program under the name `ProcessIntsIntUnaryOperator` in the *javasrc* repository.

`Default` methods in interfaces can be used to produce *mixins*, as described in Recipe 9.9.

9.3 Simplifying Processing with Streams

Problem

You want to process some data through a pipeline-like mechanism.

Solution

Use a `Stream` class and its operations.

Discussion

Streams are a new mechanism introduced with Java 8 to allow a collection to send its values out one at a time through a pipeline-like mechanism where they can be processed in various ways, with varying degrees of parallelism. There are three types of methods involved with Streams:

- Stream-producing methods (see Recipe 7.4).
- Stream-passing methods, which operate on a Stream and return a reference to it, in order to allow for *fluent programming* (chained method calls); examples include `distinct()`, `filter()`, `limit()`, `map()`, `peek()`, `sorted()`, and `unsorted()`.
- Stream-terminating methods, which conclude a streaming operation; examples include `collect()`, `count()`, `findFirst()`, `max()`, `min()`, `reduce()`, and `sum()`.

In Example 9-2, we have a list of `Hero` objects representing (mostly) heroic individuals through the ages. We use the `Stream` mechanism to filter just the adult heroes and then sum their ages. We use it again to sort the heroes' names alphabetically.

In both operations we start with a stream generator (`Arrays.stream()`); we run it through several steps, one of which involves a mapping operation. Mapping in this context, not to be confused with `java.util.Map`, transforms the stream data from one form to another as it is sent along the pipeline. An example of this is mapping a `Person` object to the person's age (see `mapToInt` in Example 9-2). The stream is wrapped up by a terminal operation. The map and filter operations almost invariably are controlled by a lambda expression (inner classes would be too tedious to use in this style of programming!).

Example 9-2. main/src/main/java/functional/SimpleStreamDemo.java

```
  static Hero[] heroes = {
    new Hero("Grelber", 21),
    new Hero("Roderick", 12),
    new Hero("Thor", 35),
    new Hero("Superman", 65),
    new Hero("Batman", 37),
    new Hero("Palladin", 50),
    new Hero("Iron Man", 42),
    new Hero("Athena", 50)
  };

  public static void main(String[] args) {

    long adultYearsExperience = Arrays.stream(heroes)
        .filter(b -> b.age >= 18)
        .mapToInt(b -> b.age)
        .sum();
    System.out.println("We're in good hands! The adult superheros have " +
        adultYearsExperience + " years of experience");

    List<Object> sorted = Arrays.stream(heroes)
        .sorted((h1, h2) -> h1.name.compareTo(h2.name))
        .map(h -> h.name)
        .collect(Collectors.toList());
    System.out.println("Heroes by name: " + sorted);
  }
```

And let's run it to be sure it works:

```
We're in good hands! The adult superheros have 300 years of experience
Heroes by name: [Athena, Batman, Grelber, Iron Man, Palladin,
    Roderick, Superman, Thor]
```

See the Javadoc for the `java.util.stream.Stream` interface for a complete list of the operations.

9.4 Simplifying Streams with Collectors

Problem

You construct Streams, but it's hard to get the results into a form you want.

Solution

Consider using a `Collector` to terminate your stream.

Discussion

Example 9-2 ended the second half with a call to `collect()`. The argument to `collect()` is of type `Collector`, which this recipe considers in more detail. `Collector`s are a form of what classical FP languages call *folds*. Folds are also called reduce, accumulate, aggregate, compress, or inject operations. A fold in functional programming is a terminal operation, analogous to collapsing a whole string of tickets into a flat pile (see Figure 9-4). The string of tickets represents the `Stream`, the folding operation is represented by a function, and the final result is, well, all folded up. It will often include a combining operation, analogous to counting the tickets as they are folded.

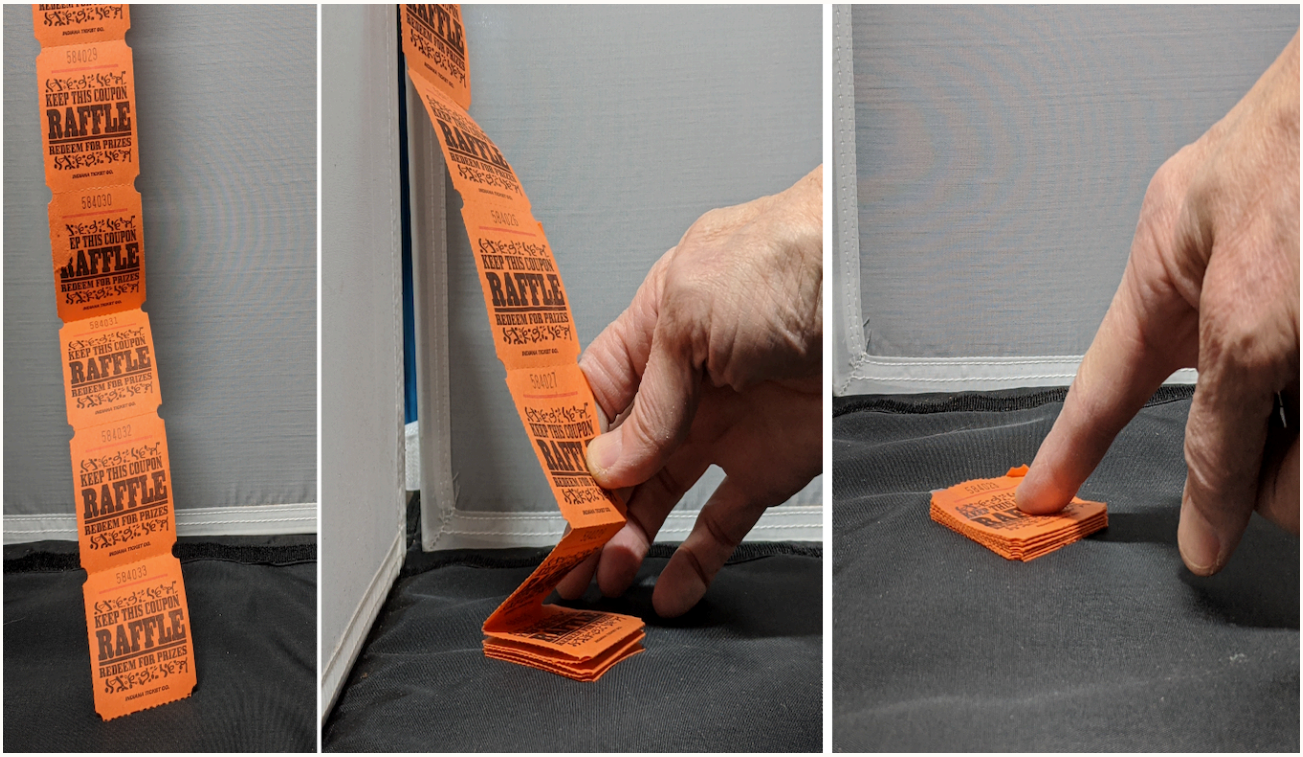


Figure 9-4. Stream of tickets before folding, during folding, and after folding: a terminal operation

Note that in the first panel of Figure 9-4 we don't know how long the `Stream` is, but we expect that it will terminate eventually.

Collector as used in Java refers to a terminal function that analyzes/summarizes the content of a `Stream`. Technically, `Collector` is an interface whose implementation is specified by three (or four) functions that work together to accumulate entries into a `Collection` or `Map` or other mutable result container, and optionally a final transform on the result. The functions are as follows:

`supplier()`

Creates a new result container

`accumulator()`

Adds a new data element into the result container

`combiner()`

Combines two result containers into one

`finisher()` *(optional)*

Performs a final transform on the container

While you can easily compose your own `Collector` implementation, it is often expedient to use one of the many useful implementations predefined in the `Collectors` class. Here are a couple of simple examples:

```
int howMany = cameraList.stream().collect(Collectors.counting());
double howMuch = cameraList.filter(desiredFilter).
      collect(Collectors.summingDouble(Camera::getPrice);
```

The `::` syntax is explained in Recipe 9.8; the effect here is to replace each `Camera` object in the Stream with the result of calling its `getPrice()` method.

In Example 9-3 I implement the classic *word frequency count* algorithm: take a text file, break it into individual words, count the occurrence of each word, and list the *n* most used words, sorted by frequency in descending order.

In Unix terms this could be implemented (assuming *n* = 20) as:

```
prep $file | sort | uniq -c | sort -nr | head -20
```

where `prep` is a script that uses the Unix tool `tr` to break lines into words and turn the words into lowercase.

Example 9-3. main/src/main/java/functional/WordFreq.java

```
/**
 * Implement word frequency count; shown in two main steps
 * (WordFreq2 shows the steps combined into one statement).
 */
public class WordFreq {
  public static void main(String[] args) throws IOException {

    String fileName = args.length == 0 ?  "README.adoc" : args[0];

    // Collect words with a mutable reduction into
    // a Map<String,Long>
    Map<String,Long> map = Files.lines(Path.of(fileName))
      .filter(line->!line.isEmpty())
      .flatMap(s -> Stream.of(s.split(" ")))
      .collect(Collectors.groupingBy(
        String::toLowerCase, Collectors.counting()));

    // Get map's entrySet as a stream,
    // sort -r, count, print highest 20.
    map.entrySet().stream()
      .sorted(Map.Entry.<String,Long>comparingByValue()
      .reversed())
      .limit(20)
      .map(entry -> "%4d %s".formatted(entry.getValue(), entry.getKey()))
      .forEach(System.out::println);
  }
}
```

There are two steps. First, create a map of the words and their frequencies. Second, sort these in reverse order, stop at number 20, format them neatly, and print.

The first part uses `Files.lines()` from Chapter 10 to get a `Stream` of `Strings`, which is broken into individual words using the `Stream` method `flatMap()` combined with the `String` method `split()` to break on one or more spaces. We use `flatmap()` here because `map()` would produce a stream of `<List<String>>`, whereas `flatmap()` produces the desired `Stream` of `Strings`. The result is collected into a map using a `Collector`. I had initially used a homemade collector:

```
    .collect(HashMap::new, (m,s)->m.put(s, m.getOrDefault(s,0)+1), HashMap::putAll);
```

This form of `collect()` takes three arguments:

- A `Supplier<R>` or factory method to create an empty container; here I'm just using the `HashMap` constructor.
- An `Accumulator` of type `BiConsumer<R,? super T>` to add each element into the map, adding one each time the same word is found.
- A `Combiner` of type `BiConsumer<R,R> combiner)` to combine all the collections used.

In the case of parallel streams (see Recipe 9.7), the `Supplier` may be called multiple times to create multiple containers, and each part of the Stream's content will be handled by one `Accumulator` into one of the containers. The `Combiner` will merge all the containers into one at the end of processing.

However, Sander Mak pointed out that it's easier to use the existing `Collectors` class's predefined collector `groupingBy`, combining the `toLowerCase()` call and the `collect()` call with this:

```
    .collect(Collectors.groupingBy(String::toLowerCase, Collectors.counting()));
```

To further simplify the code, you could combine the two statements into one by doing the following:

- Removing the return value and assignment `Map<String,Long> map =`
- Removing the semicolon from the end of the `collect` call
- Removing the `.map()` from the `entrySet()` call

Then you can say you've implemented something useful in a single Java statement!

9.5 Simplifying Streams with Stream Gatherers

22P

Problem

You are composing a `Stream` but find that the intermediate operations cannot be expressed succinctly. You realize that nothing in the material covered so far will help simplify them.

Solution

Use a `Stream Gatherer`, either one that is built-in or one of your own construction.

Discussion

The Streams API has been quite successful in allowing the creative composition of applications. As pointed out in Recipe 9.3, a stream consists of one generational step, a number of intermediate steps, and one terminal step. However, the set of medial operations is finite. Suppose we have a class full of graduate students whom we want to set to work researching some aspect of life in countries around the world. Assume that each student can handle, say, 10 countries in one term. We have the ISO 3166 two-letter country codes saved in a text file. Our goal is to print the list of 10 countries to hand out to each student.

Up to and including Java 21, there was no way to implement this in Streams without writing some extra code. However, in Java 22 and later, we can use a `Stream` with the new `Stream` `Gatherer` mechanism to do this easily, as shown in Example 9-4.

Example 9-4. main/src/main/java/functional/StreamGathererDemo.java

```
Files.lines(Path.of("country_codes.txt"))
  .gather(Gatherers.windowFixed(NUM_COUNTRIES))
  .limit(5)  // Just sample
  .forEach(System.out::println);
```

When we run this, it "just works." We stopped it at five rows just to prove that it works:

```
$ java StreamGathererDemo.java
["AA", "AB", "AC", "AD", "AE", "AF", "AG", "AH", "AI", "AJ"]
["AK", "AL", "AM", "AN", "AO", "AP", "AQ", "AR", "AS", "AT"]
["AU", "AV", "AW", "AX", "AY", "AZ", "BA", "BB", "BC", "BD"]
["BE", "BF", "BG", "BH", "BI", "BJ", "BK", "BL", "BM", "BN"]
["BO", "BP", "BQ", "BR", "BS", "BT", "BU", "BV", "BW", "BX"]
$
```

As is common, a utility class is named in the plural, e.g., `Gatherers`. There are five pre-defined gatherers, obtainable from these static methods in the `Gatherers` class:

fold

A many-to-one gatherer; see the fold operation in Recipe 9.4

mapConcurrent

Like regular `Stream.map()` but lets you specify a maximum level of concurrency, which is implemented using virtual threads (Recipe 11.2)

scan

Outputs the result of a given operation on each item in the input stream (see `StreamGathererScanDemo` in *javasrc*)

windowFixed

Groups into fixed-size buckets, as shown in Example 9-4

windowSliding

Groups into sequential permutations, e.g, given the integers from 1 up as input, `Gatherer.windowSliding(3)` would output [1,2,3], [2,3,4], [3,4,5], [4,5,6], etc.

9.6 Simplifying Streams with Your Own Stream Gatherer 22P

Problem

There doesn't seem to be a stream gatherer that does quite what you want.

Solution

Write your own! `Gatherer` is just an interface.

Discussion

You can implement your own `Stream Gatherer`. A `Gatherer` consists of up to four functions:

- The initializer function (optional) creates an object to maintain any private state needed during processing.
- The integrator function (required) processes or "integrates" each element from the input stream. It can use the private state object from the initializer function to determine whether or not to output the current input item to the output stream. It can even stop the processing before reaching the end of the input stream, if the logic so dictates.
- The combiner function (optional) can evaluate the gatherer in parallel if the input stream is parallel. This can be used in cases where an operation is ordered in nature and can't be parallelized.
- The finisher function (optional) is invoked when there are no more input elements to consume. This function can inspect the private state object and choose to emit additional (leftover?) output elements, throw an exception if requirements were not met by any or all of the input elements, or simply do nothing.

Suppose we have a list of sale amounts and want to use streams to get a list that is distinct, but only for sales that differ by a rounding error amount (less than one cent). For this we could use various stream operators, including `filter()`, but this makes a good demo of how simple a custom `Gatherer` implementation can be. Example 9-5 shows an implementation of this requirement.

Example 9-5. main/src/main/java/functional/StreamGathererCustom.java

```
// Mutable State
class DoubleHolder {
  Double value = 0.0D;
}

// Threshold for distinctness - anything over a penny is distinct
double THRESHOLD = 0.01D;

// The star of the show: a simple custom Gatherer
```

```
Gatherer<Double, DoubleHolder, Double> distinctByAmount =
  Gatherer.ofSequential(

    // Initializer
    DoubleHolder::new,

    // Integrator
    (state, element, downstream) -> {
      var distinct = Math.abs(element - state.value) > THRESHOLD;
      state.value = element;
      return distinct ? downstream.push(element) : !downstream.isRejecting();
    }
  );

// Sample data
List<Double> sales = List.of(
  125.00,
  125.01,
  125.015,
  250.00,
  75.00
);

void main() {
  System.out.println("Raw");
  sales.stream().sorted().forEach(System.out::println);

  System.out.println("Distinct");
  sales.stream()
    .sorted()
    .gather(distinctByAmount)
    .forEach(System.out::println);
}
```

The key component is the `Gatherer`, defined using the factory method `Gatherer.of()`. There are many overloads of this, but since we're only going to run a small number of transactions each time, we don't need parallelism, so we don't need the optional combiner function. And, each distinct value is pushed downstream as we process it, so we don't even need the optional finisher function. The initializer just constructs an instance of our state class (`DoubleHolder`). The integrator takes each item and compares it to the previous one; if they differ enough, the item is pushed downstream. The integrator method returns `boolean` to indicate whether more elements are to be consumed or not; if the downstream is already in rejecting mode, there is no point in pushing any more elements, so the `Gatherer` will shut down.

Note that in the definitions of gatherers, the type paramater `TR` appears; this indicates that the same type applies both to values consumed (`T` for type of input) and sent downstream (`R` for returned) by this gatherer.

See Also

There is more to the subject of `Gatherer`s than we can cover in these two recipes. See the `Gatherer` documentation. Another tutorial is on Ben Weidig's blog.

9.7 Improving Throughput with Parallel Streams and Collections

Problem

You want to combine `Stream`s with parallelism and still be able to use the non-thread-safe Collections API.

Solution

Use a parallel stream.

Discussion

To enable efficient operations, *parallel streams* let you use the non-thread-safe collections safely, as long as you do not modify the collection while you are operating on it.

To use a parallel stream, you just ask the collection for it, using `parallelStream()` instead of the `stream()` method we used in Recipe 9.3. This does not guarantee a speed-up, though it may be faster: to be sure, you need to profile the code with a reasonable-sized volume of data; see Recipe 2.16.

For example, suppose that our camera business takes off, and we need to find cameras by type and price range *quickly*, and with less code than we used before (see Example 9-6).

Example 9-6. src/main/java/functional/CameraSearchParallelStream.java

```
public static void main(String[] args) {
  System.out.println("Search Results using For Loop");
  for (Object camera : listOfCameras.parallelStream(). ❶
      filter(c -> c.isIlc() && c.price() < 500).      ❷
      toArray()) {                                    ❸
    System.out.println(camera);                       ❹
  }

  System.out.println(
    "Search Results from shorter, more functional approach");
  listOfCameras.parallelStream().                     ❺
          filter(c -> c.isIlc() && c.price() < 500).
      forEach(System.out::println);
}
```

❶ Create a parallel stream from the `List` of `Camera` objects. The end result of the stream will be iterated over by the for-each loop.

❷ Filter the cameras on price, using the same `Predicate` lambda that we used in Recipe 9.1.

❸ Terminate the `Stream` by converting it to an array.

❹ The body of the for-each loop: print one `Camera` from the `Stream`.

❺ A more concise way of writing the search.

WARNING

This is only reliable as long as no thread is modifying the data at the same time as the searching is going on. See the thread interlocking mechanisms in Chapter 11 to see how to ensure this.

9.8 Using Existing Code as Functional with

Method References

Problem

You have existing code that matches a functional interface and want to use it without renaming methods to match the interface name.

Solution

Use function references such as `MyClass::myFunc` or `someObj::someFunc`.

Discussion

The word *reference* is almost as overloaded in Java as the word *session*. Consider the following:

- Ordinary objects are usually accessed with references.
- Reference types such as `WeakReference` have defined semantics for garbage collection.
- And now, for something completely different, Java 8+ lets you reference an individual method.

The *method reference* syntax consists of an object reference or class name, two colons, and the name of a method that can be invoked in the context of the object or class name (as per the usual rules of Java, a class name can refer to static methods and an instance can refer to an instance method). To refer to a constructor as the method, you can use `new`—for example, `MyClass::new`. The reference creates a lambda that can be invoked, stored in a variable of a functional interface type, and so on.

In Example 9-7, we create a `Runnable` reference that holds, not the usual `run` method, but rather a method with the same type and arguments but with the name `walk`. Note the use of `this` as the object part of the method reference. We then pass this `Runnable` into a `Thread` constructor and start the thread, with the result that `walk` is invoked where `run` would normally be.

Example 9-7. main/src/main/java/functional/ReferencesDemo.java

```
/** "Walk, don't run" */
public class ReferencesDemo {
```

```
  // Assume this is an existing method we don't want to rename
  public void walk() {
    System.out.println("ReferencesDemo.walk(): Stand-in run method called");
  }

  // This is our main processing method; it runs "walk" in a Thread
  public void doIt() {
    Runnable r = this::walk;
    new Thread(r).start();
  }

  // The usual simple main method to start things off
  public static void main(String[] args) {
    new ReferencesDemo().doIt();
  }
}
```

The output is as follows:

```
ReferencesDemo.walk(): Stand-in run method called
```

The code could be simplified by moving the body of `doit()` into `main()`, but then the `walk()` method would need to be static, and the reference would be `ReferencesDemo::walk`.

Example 9-8 creates an `AutoCloseable` for use in a `try-with-resources`. The normal `AutoCloseable` method is `close()`, but ours is named `cloz()`. The `AutoCloseable` reference variable `autoCloseable` is created inside the `try` statement, so its `close`-like method will be called when the body completes. In this example, we are in a static `main` method wherein we have a reference `rnd2` to an instance of the class, so we use this in referring to the `AutoCloseable`-compatible method.

Example 9-8. main/src/main/java/functional/ReferencesDemo2.java

```
public class ReferencesDemo2 {
  void cloz() {
    System.out.println("Stand-in close() method called");
  }

  public static void main(String[] args) throws Exception {
    ReferencesDemo2 rd2 = new ReferencesDemo2();
```

```
    // Use a method reference to assign the AutoCloseable interface
    // variable "ac" to the matching method signature "c" (obviously
    // short for close, but just to show the method name isn't what matters).
    try (AutoCloseable autoCloseable = rd2::cloz) {
      System.out.println("Some action happening here.");
    }
  }
}
```

The output is as follows:

```
Some action happening here.
Stand-in close() method called
```

It is, of course, possible to use this with your own functional interfaces, as defined in Recipe 9.2. You're also probably at least vaguely aware that any normal Java object reference can be passed to `System.out.println()` and you'll get some description of the referenced object. Example 9-9 explores these two themes. We define a functional interface imaginatively known as `FunInterface` with a method with a bunch of arguments (merely to avoid it being mistaken for any existing functional interface). The method name is `process`, but as you now know the name is not important; our implementation method goes by the name `work`. The `work` method is static, so we could not state that the class implements `FunInterface` (even if the method names were the same, a static method may not hide an inherited instance method), but we can nonetheless create a lambda reference to the `work` method. We then print this out to show that it has a valid structure as a Java object.

Example 9-9. main/src/main/java/functional/ReferencesDemo3.java

```
public class ReferencesDemo3 {

  interface FunInterface {
    void process(int i, String j, char c, double d);
  }

  public static void work(int i, String j, char c, double d){
    System.out.println("In work " + c);
  }

  public static void main(String[] args) {
```

```
    FunInterface sample = ReferencesDemo3::work;
    sample.process(42, "Hello", '\u263A', Math.PI);
    System.out.println("My process method is " + sample);
  }
}
```

This generates the following output:

```
In work ☺
My process method is functional.ReferencesDemo3$$Lambda/0x00000cde2414c3f8@6591f517
```

The `$Lambda` in the name is structurally similar to the `$1` used for anonymous inner classes.

The fourth way, to use a function reference, an instance method of an arbitrary object of a particular type, may be the most esoteric. It allows you to declare a reference to an instance method but without specifying which instance. Because there is no particular instance in mind, you again use the class name. This means you can use it with any instance of the given class! In Example 9-10, we have an array of `String`s to sort. Because the names in this array can begin with a lowercase letter, we want to sort them using the `String` method `compareToIgnoreCase()`, which nicely ignores case differences for us.

Because I want to show the sorting several different ways, I set up two array references: the original, unsorted reference, and a working reference that is re-created, sorted, and printed using a simple dump routine, which isn't shown (it's just a `for` loop printing the strings from the passed array).

Example 9-10. main/src/main/java/functional/ReferencesDemo4.java

```
  static final List<String> unsortedNames = List.of(
    "Gosling", "de Raadt", "Amdahl", "Turing", "Ritchie", "Hopper"
  );

  public static void main(String[] args) {
    List<String> names;

    // Sort using
    // "an Instance Method of an Arbitrary Object of a Particular Type"
    names = new ArrayList<>(unsortedNames);
    Collections.sort(names, String::compareToIgnoreCase);  ❶
    dump(names);
```

```
    // Equivalent Lambda:
    names = new ArrayList<>(unsortedNames);
    Collections.sort(names,
      (str1, str2) -> str1.compareToIgnoreCase(str2));     ❷
    dump(names);

    // Equivalent old way:
    names = new ArrayList<>(unsortedNames);
    Collections.sort(names, new Comparator<String>() {     ❸
      @Override
      public int compare(String str1, String str2) {
        return str1.compareToIgnoreCase(str2);
      }
    });
    dump(names);

    // Simpest way, using existing comparator
    names = new ArrayList<>(unsortedNames);
    Collections.sort(names, String.CASE_INSENSITIVE_ORDER); ❹
    dump(names);
  }
```

❶ Using an instance method of an arbitrary object of a particular type declares a reference to the `compareToIgnoreCase` method of any `String` used in the invocation.

❷ Shows the equivalent lambda expression.

❸ Shows "Your grandparents' Java" way of doing things.

❹ Using the exported `Comparator` directly, just to show that there is always more than one way to do things.

Just to be safe, I ran the demo and got the expected output:

```
Amdahl, de Raadt, Gosling, Hopper, Ritchie, Turing
Amdahl, de Raadt, Gosling, Hopper, Ritchie, Turing
Amdahl, de Raadt, Gosling, Hopper, Ritchie, Turing
Amdahl, de Raadt, Gosling, Hopper, Ritchie, Turing
```

9.9 Java Mixins: Mixing in Methods

Problem

You've heard about mixins and want to apply them in Java.

Solution

Use static imports. Or, declare one or more functional interfaces with a `default` method containing the code to execute, and simply implement it.

Discussion

Developers from other languages sometimes deride Java for its inability to handle mix-ins, the ability to mix in bits of code from other classes.

One way to implement mixins is with the *static import* feature, which has been in the language for a decade. This is often done in unit testing (see Recipe 2.9). A limitation of this approach is that, as the name implies, the methods must be static methods, not instance methods.

A newer mechanism depends on an interesting bit of fallout from the Java 8 language changes in support of lambdas: you can now mix in code from unrelated places into one class. Has Java finally abandoned its staunch opposition to multiple inheritance? It may seem that way when you first hear it, but relax: you can only pull methods from multiple interfaces, not from multiple classes. If you didn't know that you could have methods defined (rather than merely declared) in interfaces, see "Subclass, Abstract Class, or Interface?". Consider the following example:

main/src/main/java/lang/MixinsDemo.java

```
interface Bar {
  default String filter(String s) {
    return "Filtered " + s;
  }
}

interface Foo {
  default String convolve(String s) {
    return "Convolved " + s;
```

```
  }
}

public class MixinsDemo implements Foo, Bar{

  public static void main(String[] args) {
    String input = args.length > 0 ? args[0] : "Hello";
    String output = new MixinsDemo().process(input);
    System.out.println(output);
  }

  private String process(String s) {
    return filter(convolve(s)); // methods mixed in!
  }
}
```

If we run this, we see the expected results:

```
C:\javasrc>javac -d build lang/MixinsDemo.java
C:\javasrc>java -cp build lang.MixinsDemo
Filtered Convolved Hello

C:\javasrc>
```

Presto—Java now supports mixins! And if there are conflicting methods, please refer back to "Default methods".

Does this mean you should go crazy trying to build interfaces with code in them? No. Remember this mechanism was designed to do the following:

- Provide the notion of functional interfaces for use in lambda calculations.
- Give the ability to retrofit interfaces with new methods, without having to change *old* implementations. As with many changes made in Java over the years, backward compatibility was a huge driver.

Used sparingly, functional interfaces can provide the ability to mix in code to build up applications in another way than direct inheritance, aggregation, or AOP. Overused, it can make your code heavy, drive pre-Java 8 developers crazy, and lead to chaos.

9.10 Functional Programming with Flow and Reactive Streams

Problem

You have a lot of different I/O that could be handled asynchronously, and you want to try working with the *reactive* style of programming.

Solution

Use the Reactive API in `java.util.concurrent.Flow`, or use a third-party implementation.

Discussion

The original Reactive Manifesto states that reactive systems "are software architectures that are responsive, resilient, elastic and message driven." The last attribute implies a publish-subscribe model; the former three imply a form in which one (or more) publishers push data to one or more subscribers, each of whom can control how rapidly data gets sent to them. This notion of controlling the data rate is often called *back pressure*, in the sense that it exerts pressure on the sender to reduce the flow. Back pressure is new and significant, as it can allow the receiver to inform the sender when the latter is sending too quickly, for example. In this API, back pressure is controlled very simply, by the subscriber client code calling a `request()` method on the `Subscriber`, both on connection and after every receipt of data.

The class `java.util.concurrent.Flow` exports two interfaces, called `Publisher` and `Subscriber`. `Subscriber` can be implemented directly. The `Publisher` has more to do, and users of this API will often use the `SubmissionPublisher` implementation class directly. Example 9-11 shows this, with one `Publisher` and two `Subscriber`s.

Example 9-11. main/src/main/java/functional/FlowDemo.java

```
public class FlowDemo {

  public static void main(String[] args) throws InterruptedException {

    // This is a Flow.Publisher implementation that handles submission/
    // forwarding, and has a close() method.
    SubmissionPublisher<String> publisher = new SubmissionPublisher<>();
```

```
  // Our implementation of the Subscriber interface
  class MySubscriber implements Flow.Subscriber<String> {
    private Flow.Subscription subscription;
    private String id;

    MySubscriber(String id) {
      this.id = id;
    }

    @Override
    public void onSubscribe(Flow.Subscription subscription) {
      this.subscription = subscription;
      subscription.request(getNumBuffers());
    }

    @Override
    public void onNext(String item) {
      System.out.println(id + ": Received: " + item);
      subscription.request(getNumBuffers());
    }

    @Override
    public void onError(Throwable throwable) {
      throwable.printStackTrace();
    }

    @Override
    public void onComplete() {
      System.out.println(id + ": Done");
    }
  };
  Flow.Subscriber<String> subscriber = new MySubscriber("1");
  publisher.subscribe(subscriber);
  Flow.Subscriber<String> subscriber2= new MySubscriber("2");
  publisher.subscribe(subscriber2);
  for (String mesg : messages) {
    System.out.println("Sending: " + mesg);
    publisher.submit(mesg);
  }
  publisher.close();
  Thread.sleep(1000);
}

/**
 * Compute the number of buffers we want to receive, to
```

```
   * manage "back pressure" - so we don't get sent more than
   * we can handle.
   * A placeholder for now - "1" will always work.
   */
  static int getNumBuffers() {
    return 1;
  }

  static String[] messages = {
    "Hello, world of Java!",
    "Hope you're having a good day!",
    "Things are really hopping now",
    "The clock is ticking.",
    "Day is done. Goodbye!",
  };
}
```

Here is the output of running Example 9-11:

```
$ java FlowDemo.java
Sending: Hello, world of Java!
Sending: Hope you're having a good day!
Sending: Things are really hopping now
Sending: The clock is ticking.
Sending: Day is done. Goodbye!
2: Received: Hello, world of Java!
1: Received: Hello, world of Java!
2: Received: Hope you're having a good day!
1: Received: Hope you're having a good day!
2: Received: Things are really hopping now
1: Received: Things are really hopping now
2: Received: The clock is ticking.
1: Received: The clock is ticking.
2: Received: Day is done. Goodbye!
1: Received: Day is done. Goodbye!
1: Done
2: Done
$
```

See Also

There is much more to the world of reactive programming. Besides the Java 9 `Flow` described here, RxJava and Akka-Streams are two more comprehensive Java application

frameworks. Android users can combine RxJava and RxAndroid. Flutter developers can use RxDart. React Native is a complete JavaScript-based toolkit for building reactive apps for desktop and mobile. Baeldung's website and the article "Reactive Programming with Java 9's Flow" by Moisés Macero both contain extended Java examples.

1 Haskell was used to write a fairly complete Twitter clone in a few hundred lines; see simple-twitter on GitHub.

2 If you ever have to build this kind of service where the data is stored in a relational database using the Java Persistence API (JPA), you should check out the Spring Data or Jakarta Data frameworks. These allow you to define an `interface` with method names like `findCameraByInterchangeableTrueAndPriceLessThan(double price)` and have the framework implement these methods for you.

3 If you're just not that into cameras, the description "Interchangeable Lens Camera (ILC)" includes two categories of what you might find in a camera store: traditional DSLR (Digital Single Lens Reflex) cameras and the newer category of Mirrorless Cameras like the Nikon Z-30/Z50, Sony ILCE (formerly known as NEX), and the Canon EOS-M, all of which are smaller and lighter than the older DSLRs. Both types allow the use of different lenses, usually through a twist-lock mounting system that holds the lens in place on the camera body.

Previous chapter
8. Object-Oriented Techniques

Next chapter
10. Input and Output: Reading, Writing, and Directory Tricks